Linux External

Q) Explain the difference features of linux Operating System.

Linux is a free, open-source, and powerful operating system used on servers, desktops, and even mobile devices. It is based on UNIX and is known for its stability, security, and flexibility.

Here are the main features of the Linux Operating System: ------

1. Open Source

- Linux is open source, which means its source code is freely available to everyone.
- Anyone can read, modify, and distribute the code.
- This helps developers improve Linux and create custom versions for different needs.

2. Free to Use

- Linux is completely free to download and install.
- There is no need to buy a license, unlike Windows or macOS.
- Even big companies use Linux to save money on operating systems.

3. Multiuser System

- Linux allows many users to use the system at the same time.
- Each user gets their own space and settings.
- Users cannot interfere with each other's files unless given permission.

4. Multitasking

- Linux can run many tasks at the same time.
- For example, you can play music, download a file, and type in a document all at once.
- It manages tasks efficiently without slowing down.

5. Security

- Linux is very secure and hard to hack.
- It uses file permissions, user authentication, and firewalls for protection.
- Viruses are rare in Linux, which is why it is used in servers and banking systems.

6. Portability

- Linux can run on different types of hardware like PCs, laptops, smartphones, routers, and even supercomputers.
- It works well on both old and modern machines.

7. Shell/Command Line Interface (CLI)

- Linux provides a powerful command-line interface (called the shell).
- Users can give commands to the system directly using the terminal.
- It is useful for automation, programming, and system control.

8. File System Support

- Linux supports many types of file systems like ext3, ext4, FAT32, NTFS, etc.
- It can also read/write data from other OS file systems (like Windows).

9. Customization

- Linux is highly customizable.
- Users can change the desktop environment, theme, boot screen, and more.
- Developers can even remove unwanted parts and create a lightweight version.

10. Community Support

- There is a large community of Linux users and developers.
- Help is available through forums, websites, and online documentation.
- Even beginners can solve problems easily with community support.

Q) Write 10 commands of linux operating system.

Linux provides many powerful commands to manage files, users, processes, and system operations. These commands are typed in the **terminal** and help users control the system easily.

Here are **10 important Linux commands** with simple explanations :----

1. ls –> List Files and Directories :

- Shows the list of files and folders in the current directory.

Example:

```
ls
```

2. pwd –> Print Working Directory :

- Displays the **current location** (path) in the file system.

Example:

```
pwd
```

Output:
/home/user/Documents

3. cd –> Change Directory :

- Used to **move** from one directory to another.

Example:

```
cd /home/user/Desktop
```

4. mkdir –> Make a New Directory :

- Creates a new folder (directory).

Example:

```
mkdir myfolder
```

5. rm – Remove Files or Directories :

- Deletes files or folders permanently.

Example (file):

```
rm myfile.txt
```

Example (folder):

```
rm -r myfolder
```

6. cp – Copy Files or Directories :

- Copies files or folders from one place to another.

Example:

```
cp file1.txt /home/user/Desktop/
```

7. mv – Move or Rename Files :

- Moves a file/folder or renames it.

Example (move):

```
mv file1.txt /home/user/Documents/
```

Example (rename):

```
mv oldname.txt newname.txt
```

8. touch – Create an Empty File :

- Makes a new, empty file.

Example:

```
touch newfile.txt
```

9. cat – Display File Content :

- Shows the contents of a text file on the terminal.

Example:

```
cat notes.txt
```

10. clear – Clear the Terminal Screen :

- Clears all text from the terminal for a clean view.

Example:

clear

Q) Explain what are the main difference between linux and unix.

Linux and Unix are both powerful operating systems. They are similar in many ways but also have key differences.
Unix is the original operating system, and **Linux** is a newer version that is inspired by Unix.

Let's understand the main differences between Linux and Unix in simple words ::---

1. Origin

- **Unix:**
  Developed in the 1970s at **AT&T Bell Labs**. It is the **original** OS from which Linux was inspired.
- **Linux:**
  Created in **1991 by Linus Torvalds**. It is a **free and open-source** version of Unix.

2. Cost

- **Unix:**
  Mostly **paid**. You need to **buy a license** to use it (e.g., IBM AIX, HP-UX).
- **Linux:**
  Free and open-source. Anyone can download, use, and modify it.

3. Source Code Availability

- **Unix:**
  Source code is **closed** (not available to public) for most versions.
- **Linux:**
  Source code is **open**, which means anyone can view and change it.

4. Usage

- **Unix:**
  Used mostly in **large servers, mainframes, and special systems** like banking and telecom.

- **Linux:**
  Used in **personal computers, servers, smartphones (like Android), IoT devices**, and even supercomputers.

5. Hardware Support

- **Unix:**
  Supports **specific hardware** (e.g., works only on Sun, IBM, or HP machines).
- **Linux:**
  Supports **almost all hardware** (PCs, laptops, mobile devices, etc.).

6. User Interface

- **Unix:**
  Mostly **command-line interface**. Some versions have limited graphical user interface (GUI).
- **Linux:**
  Has both **command-line and user-friendly GUI** (like GNOME, KDE, etc.).

7. Development and Community

- **Unix:**
  Developed by **specific companies**, with limited community support.
- **Linux:**
  Developed by **community and companies together**. Has **huge online support** and forums.

8. File System

- **Unix:**
  Uses file systems like **jfs, ufs, zfs**, etc.
- **Linux:**
  Uses file systems like **ext2, ext3, ext4, Btrfs**, etc.

9. Examples

- **Unix:**
  IBM AIX, HP-UX, Solaris, BSD.

- **Linux:**
  Ubuntu, Fedora, Debian, Red Hat, Arch Linux.

10. Licensing

- **Unix:**
  Licensed by individual companies and is **not free**.
- **Linux:**
  Uses **GPL (General Public License)**, which allows free use and distribution.

Q) What the commands does :

(a) cmp (b) diff (c)comm (d) cut (e) paste.

In Linux, many commands are used to compare and manipulate text files. The following five commands are used for **comparing**, **cutting**, and **pasting** text from files.

Let's understand each command one by one :----

(a) cmp Command – (Compare Two Files Byte by Byte)

- The cmp command compares two files **byte by byte**.
- It tells you **where the first difference** occurs.
- If both files are the same, it shows **no output**.

Syntax:

```
cmp file1.txt file2.txt
```

Example Output:

```
file1.txt file2.txt differ: byte 7, line 1
```

(b) diff Command – (Show Line-by-Line Differences)

- The diff command compares two text files **line by line**.
- It shows **which lines are different** and **how to change** one file to make it like the other.
- It is helpful in programming to check code changes.

Syntax:

diff file1.txt file2.txt

Example Output:

1c1

< Hello world

> Hello Linux

(c) comm Command – (Compare Sorted Files Line by Line)

- The comm command compares **two sorted files** and shows:
  - Lines only in **file1**
  - Lines only in **file2**
  - Lines **common** to both files
- The output is divided into **three columns**.

Syntax:

comm file1.txt file2.txt

Note: Files must be **sorted** before using comm.

Example Output:

apple

banana

cherry

- First column: only in file1
- Second column: only in file2
- Third column: in both

(d) cut Command – (Cut and Extract Columns or Fields)

- The cut command is used to **cut or extract** specific columns or fields from a file.
- It is useful when working with **CSV files** or **log files**.

Syntax (by character position):

```
cut -c 1-5 filename.txt
```

Syntax (by field with delimiter):

```
cut -d ',' -f1,3 filename.csv
```

- -d specifies the delimiter (e.g., comma)
- -f selects the field number

(e) paste Command – (Merge Lines from Two Files)

- The paste command **joins lines** of two or more files **side by side**.
- It puts the contents of the files in **parallel columns**.

Syntax:

```
paste file1.txt file2.txt
```

Example Output:

```
apple   red
banana  yellow
grape   purple
```

Q) Discuss the following :

(a) AWK used in system variable

(b) File Handling

(1) AWK Used in System Variable

AWK :--

- **AWK** is a **powerful text-processing tool** in Linux.
- It is used to **search, filter, and process** text files or output line by line.
- It's especially useful for working with **system variables**, logs, or output of commands.

AWK with System Variables

- Linux system variables like **$PATH, $HOME, $USER, etc.** store important system information.
- AWK can be used to **process and analyze** the output that includes these variables.

Example 1: Print System Variable

```
echo $PATH | awk -F ':' '{for(i=1; i<=NF; i++) print $i}'
```

Explanation:

- $PATH contains many directories separated by :
- awk -F ':' sets the separator as :
- The for loop prints each directory in a new line

Example 2: Use AWK to Filter Command Output with Variables

```
ps aux | awk '$1 == "root" {print $0}'
```

Explanation:

- This command prints all processes run by the **root** user using AWK.

Example 3: Print System Info

```
df -h | awk '{print $1, $5}'
```

Explanation:

- df -h shows disk usage.
- This AWK command prints only the **file system name** and **used space**.
-

AWK is Useful For:

- Filtering logs.
- Formatting data.
- Extracting system variable values.
- Automating reports.

(2) File Handling in Linux

File Handling :-

- File handling in Linux means **creating, viewing, modifying, copying, moving, and deleting files** using commands.
- It helps users and systems manage data stored in files.

Common File Handling Commands

Command	Purpose
touch	Create an empty file.
cat	View contents of a file.
cp	Copy a file.
mv	Move or rename a file.
rm	Delete a file.
nano or vi	Edit a file using a text editor.

Examples of File Handling

1. Create a file

```
touch myfile.txt
```

2. View content of file

```
cat myfile.txt
```

3. Copy file

```
cp myfile.txt backup.txt
```

4. Move/Rename file

```
mv myfile.txt documents/
```

5. Delete file

```
rm backup.txt
```

Using Redirection Operators for File Handling

- > → Write to file (overwrite)
- >> → Append to file

Example:

```
echo "Hello World" > file.txt            # Creates or overwrites
echo "Another line" >> file.txt          # Appends to file
```

Q) Write a shell program for counting characters, words and line.

```
#!/bin/bash

# Ask the user to enter the filename      → This is comment
echo "Enter the filename:"
read filename

# Check if file exists                    → This is comment
if [ -f "$filename" ]; then
   # Use wc command to count
   echo "Counting in file: $filename"
   echo "----------------------------"
   echo "Number of lines     : $(wc -l < $filename)"
   echo "Number of words     : $(wc -w < $filename)"
   echo "Number of characters: $(wc -c < $filename)"
else
   echo "File not found!"
fi
```

Explanation (in simple words):

1. #!/bin/bash → This line tells the system it's a shell script.

2. read filename → Takes file name from the user.
3. if [-f "$filename"] → Checks if the file exists.
4. wc -l → Counts number of **lines**.
5. wc -w → Counts number of **words**.
6. wc -c → Counts number of **characters**.
7. < $filename → Sends the file as input to wc.

Sample Output:

Enter the filename:

test.txt

Counting in file: test.txt

Number of lines : 5

Number of words : 20

Number of characters: 125

Q) What is shell? What are the its types?

Shell :--

- In Linux, **Shell is a program** that acts as a **bridge between the user and the operating system**.
- It takes **commands from the user**, sends them to the system to execute, and then shows the output.

Simple Example:

When you type this command in the terminal:

" ls "

- The **Shell** reads this command.
- Sends it to the **kernel**.

- The **kernel** executes it and gives the result.
- Shell displays the **list of files** on the screen.

Functions of Shell:

1. Takes **commands** from the user.
2. Sends them to the system (kernel).
3. Displays the **output**.
4. Supports **scripting** (you can write programs using shell).
5. Allows automation using **shell scripts**.

Types of Shell in Linux

There are different types of shells available in Linux. Each has its own features, but they all do the same basic job: **take input, run commands, and show output**.

Here are the **main types of shells**:

1. Bourne Shell (sh)

- **Original Unix shell** developed by **Stephen Bourne**.
- Very simple, used for scripting.
- Doesn't support advanced features like history, auto-complete.
- Path: /bin/sh

2. Bourne Again Shell (bash)

- Most **popular** and **widely used shell**.
- Improved version of Bourne Shell.
- Supports:
    - Command history
    - Auto-completion
    - Better scripting features
- Default in many Linux systems.

- Path: /bin/bash

3. C Shell (csh)

- Syntax is like the **C programming language**.
- Supports:
  - Aliases
  - Job control
- Developed by **Bill Joy**.
- Path: /bin/csh

4. Korn Shell (ksh)

- Developed by **David Korn**.
- Combines features of Bourne and C Shell.
- Better for programming.
- Supports arrays, functions, job control.
- Path: /bin/ksh

5. Z Shell (zsh)

- Advanced and user-friendly shell.
- Combines features of bash, csh, and ksh.
- Has:
  - Theme support
  - Auto-suggestions
  - Plugin support
- Popular with developers and power users.
- Path: /bin/zsh

Q) Explain the process control of linux system. Write different steps of process management in linux.

Process :-

- A **process** is a program that is currently **running** in the system.
- For example, when you open a browser, text editor, or run any command, a new **process** starts.
- Linux handles many processes at the same time using **process control and management**.

What is Process Control?

- **Process control** means managing how processes are:
  - Created
  - Run
  - Stopped
  - Paused
  - Resumed
  - Ended
- It ensures that all programs run smoothly and system resources (like CPU and memory) are used properly.

Steps of Process Management in Linux

Here are the **main steps** involved in **Linux process management** :--

- **1. Process Creation**
  - Linux creates a process using the **fork()** system call.
  - A **child process** is created from a **parent process**.
  - Both can run different tasks.
  - Example:

```
gedit &
```

2. Process Scheduling

- Linux uses a **scheduler** to decide which process should run next.
- Each process gets some time to use the **CPU**.
- Important terms:
  - **Priority** – Higher priority gets more CPU time.
  - **nice/renice** – Used to change the priority.

3. Process Execution

- When a process is running, it can be in different **states**:
  - **Running** – Actively using CPU.
  - **Waiting/Sleeping** – Waiting for resources or input.
  - **Stopped** – Paused by the user.
  - **Zombie** – Process is dead but not removed yet.

4. Process Communication

- Sometimes, processes need to **share data** or **communicate**.
- This is done using:
  - **Pipes**
  - **Signals**
  - **Shared memory**
  - **Message queues**

5. Process Termination

- A process ends when:
  - Its task is completed.
  - It is stopped by the user or system.
- Commands used:
  - kill PID
  - kill -9 PID (force kill)

- After termination, Linux frees the resources used by the process.

Useful Commands for Process Control

Command	Use
ps	Show running processes
top	Live monitoring of processes
kill PID	Stop a process
nice	Start process with priority
renice	Change priority of running process
jobs	Show background jobs
fg	Bring job to foreground
bg	Send job to background

✅ **Example: Process Control Commands**

```
sleep 100 &      # Run in background
jobs             # Show background jobs
fg               # Bring to foreground
kill %1          # Kill the first job
```